HW1 Problem 1: Lean-Into-Corner Controller

Gain Scheduling Baseline + ML Gain Scheduler Comparison

Kshitij Gupta

Feb 9, 2026

Overview (What this report covers)

This report answers **Problem 1 (Lean-Into-Corner Controller)**:

- (a) A **gain-scheduled controller** that tracks a comfort-based lean angle and produces the required plots.
- (b) **Performance profiling** and runtime considerations.
- (c) A comparison of a **classical gain scheduler** against an **ML-based gain scheduler** (trained offline), and a discussion on whether ML can compete.

1 Problem Setup and Modeling Assumptions

1.1 Goal: comfort-first lean

When a vehicle turns with lateral acceleration a_y, passengers feel a lateral inertial effect unless the cabin/vehicle leans. A comfort-first approach chooses a desired roll angle ϕ_{des} such that gravity provides a component that cancels the turning inertial effect (in the vehicle body frame).

1.2 Roll (lean) dynamics model

We use a single-DOF roll model with linear stiffness and damping:

$$I_\phi \ddot{\phi} + c_\phi \dot{\phi} + k_\phi \phi = \tau_{\text{act}} \tag{1}$$

where:

- ϕ is roll angle (rad), $\dot{\phi}$ is roll rate (rad/s)
- I_ϕ is roll inertia
- k_ϕ is roll stiffness and c_ϕ is roll damping
- τ_{act} is the actuator roll torque

Parameter values (used in simulation).

Parameter	Value	Units
I_ϕ	600	kg·m^2
k_ϕ	10000	N·m/rad
c_ϕ	2000	N·m·s/rad
g	9.81	m/s^2

1.3 Modeling and Control Assumptions

The following assumptions are made in this work:

- The vehicle roll dynamics are approximated as a single-degree-of-freedom linear system.
- Roll stiffness and damping are constant and known.
- Actuator torque is applied directly to the roll DOF (no actuator dynamics or delays).
- Vehicle speed and path curvature are known and smooth.
- The desired lean angle is computed assuming quasi-static balance between gravity and lateral acceleration.
- Passenger felt lateral acceleration is approximated by a reduced model proportional to roll tracking error.
- Tire dynamics, load transfer, and lateral slip are not explicitly modeled.
- For the ML scheduler, training is performed offline and the learned gains are bounded for safety.

2 Turn Profile and Desired Lean Angle

2.1 Curvature and lateral acceleration

For a path curvature κ (1/m) and speed v (m/s), lateral acceleration is:

$$a_y = v^2\kappa \tag{2}$$

We use a smooth curvature ramp-in/out profile to avoid actuator jerk and improve passenger comfort.

2.2 Smooth ramp-in / ramp-out curvature

We define a scalar ramp $s(t) \in [0, 1]$ using a C^1 smoothstep:

$$\text{smoothstep}(x) = 3x^2 - 2x^3, \quad x \in [0, 1] \tag{3}$$

Then for a signed turn radius R (m), curvature is:

$$\kappa(t) = \frac{s(t)}{R} \tag{4}$$

In our runs, the turn begins at $t = 1\,\text{s}$, ramps in for $1\,\text{s}$, holds, then ramps out so that the turn ends at $t = 10\,\text{s}$ (i.e., curvature returns to zero by $t = 10$).

2.3 Comfort-based desired lean angle

The comfort-first desired roll angle is:

$$\phi_{\text{des}} = -\arctan\left(\frac{a_y}{g}\right) \tag{5}$$

This is clamped to a safety limit $|\phi_{\text{des}}| \leq \phi_{\max}$, where $\phi_{\max} = 10\,\text{deg}$.

3 Controller Design (Comfort-First)

3.1 High-level structure

We use a two-loop structure:

- Outer loop: roll angle tracking $\rightarrow$ produces roll-rate command $\dot{\phi}_{\text{cmd}}$ via PI.
- Inner loop: roll-rate servo $\rightarrow$ produces feedback torque τ_{fb}.
- Feedforward torque: compensates roll spring torque at the desired lean.

3.2 Outer PI: angle error to rate command

Let $e = \phi_{\text{des}} - \phi$ and $e_I = \int e\,dt$. The outer loop computes:

$$\dot{\phi}_{\text{cmd}} = K_p e + K_i e_I \tag{6}$$

A comfort-based saturation limits the commanded rate:

$$|\dot{\phi}_{\text{cmd}}| \leq \dot{\phi}_{\text{cmd,max}}$$

and conditional integration (anti-windup) is used so the integrator does not grow when saturated.

3.3 Inner rate servo: rate error to feedback torque

The inner loop produces a damping-like torque:

$$\tau_{\text{fb}} = K_{\text{rate}}\left(\dot{\phi}_{\text{cmd}} - \dot{\phi}\right) \tag{7}$$

3.4 Feedforward torque (steady-state holding torque)

At steady state (constant $\phi \approx \phi_{\text{des}}$, $\dot{\phi} \approx 0$), the roll spring produces a restoring torque $k_\phi \phi$. To hold the desired lean without relying on feedback, we add:

$$\tau_{\text{ff}} = k_\phi \phi_{\text{des}} \tag{8}$$

Physical interpretation: this is the actuator torque required to balance the suspension/roll stiffness at the leaned equilibrium. It does *not* directly "cancel lateral acceleration"; rather, the *lean angle* changes the direction of gravity in the body frame, which reduces the lateral acceleration felt by the passenger.

3.5 Total actuator torque

$$\tau_{\text{act}} = \tau_{\text{ff}} + \tau_{\text{fb}} \tag{9}$$

4 Block Diagram and Free-Body Diagram (FBD)

4.1 Control system block diagram

Figure 1 shows the controller structure used in both the gain-scheduled baseline and the ML-based scheduler (only the mapping to gains differs).

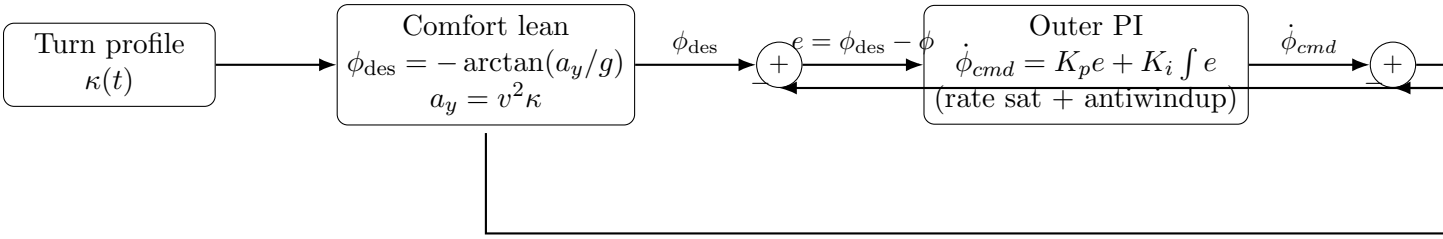
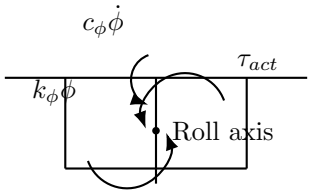


Figure 1: Block diagram of the comfort-first lean controller. Gain scheduling (table-based) or ML-based scheduling changes how (K_p, K_{rate}) are selected.

4.2 Roll-axis free-body diagram (torque balance)

A simplified torque balance about the roll axis is shown in Figure 2. The actuator torque must overcome the roll spring and damper to achieve the desired lean motion.



Dynamics: $I_\phi\ddot{\phi} = \tau_{act} - c_\phi\dot{\phi} - k_\phi\phi$

Figure 2: Simplified roll-axis torque balance (FBD). The feedforward term $\tau_{ff} = k_\phi\phi_{des}$ compensates the spring torque at the desired lean.

5 (a) Gain Scheduling Algorithm (Baseline)

5.1 Why gain scheduling?

As speed v and curvature magnitude $|\kappa|$ increase, the desired lean changes faster and the controller must respond more strongly to maintain comfort. Fixed gains can work for a single operating point but may be too sluggish or too aggressive elsewhere. Therefore, we schedule K_p and K_{rate} as smooth functions of $(v, |\kappa|)$ while keeping K_i constant.

5.2 2D scheduling with bilinear interpolation

We define grids:

$$v \in \{5, 10, 20\}\,\text{m/s}, \quad |\kappa| \in \left\{0, \frac{1}{400}, \frac{1}{200}, \frac{1}{100}\right\}\,\text{m}^{-1}$$

with lookup tables $K_p(v, |\kappa|)$ and $K_{rate}(v, |\kappa|)$. For intermediate values, we use bilinear interpolation to avoid jumps.

5.3 Pseudocode for scheduling + control loop

Listing 1: High-level pseudocode for the gain-scheduled controller.

```
for each time step t:
    kappa(t) = curvature_profile(t)          # smooth ramp in/out
    ay = v^2 * kappa(t)
    phi_des = -atan(ay/g)                    # comfort lean

    # schedule gains smoothly using bilinear interpolation
    Kp    = bilinear(v, abs(kappa), v_grid, kappa_grid, Kp_table)
    Krate = bilinear(v, abs(kappa), v_grid, kappa_grid, Krate_table)

    # outer PI -> roll rate command (with saturation + anti-windup)
    e = phi_des - phi
    phi_dot_cmd = sat( Kp*e + Ki*e_int, +/-phi_dot_cmd_max )
    if not saturated: e_int += e*dt

    # feedforward + feedback torque
    tau_ff = kphi * phi_des
    tau_fb = Krate * (phi_dot_cmd - phi_dot)
    tau_act = tau_ff + tau_fb

    # plant integration
    phi_ddot = (tau_act - cphi*phi_dot - kphi*phi) / Iphi
    phi_dot += phi_ddot * dt
    phi     += phi_dot * dt

    # comfort metric
    a_lat_feel = g*(phi_des - phi)
```

5.4 Comfort metric

We use the reduced-model comfort metric:

$$a_{\text{lat,feel}} \approx g\,(\phi_{\text{des}} - \phi) \tag{10}$$

This is zero when $\phi = \phi_{\text{des}}$. During transients, tracking error causes non-zero felt lateral acceleration.

6 (b) Performance Profiling

6.1 What dominates runtime?

For a time step size $\Delta t = 0.001\,\text{s}$ over $T = 15\,\text{s}$, each simulation run executes $\sim 15{,}000$ iterations. The dominant cost per step is:

- computing $\arctan(\cdot)$ for ϕ_{des}

- interpolation for scheduled gains (a few comparisons + lerps)
- scalar plant update (constant-time arithmetic)

6.2 Expected complexity

Each scenario is $O(N)$ where $N = T/\Delta t$. For multi-scenario evaluation (e.g., ML training), runtime scales linearly with number of scenarios.

6.3 Practical profiling (Python)

In Python, a typical approach is to use `time.perf_counter()` around the simulation function or `cProfile`. Common optimizations:

- reduce arctan calls via vectorization (optional)
- precompute curvature profile $s(t)$
- replace Python loops with NumPy vector operations (harder here due to recursion in dynamics)

In real-time deployment, gain scheduling is extremely lightweight; the ML approach also becomes lightweight after training (only a small NN forward pass).

7 Results: Gain-Scheduled Baseline (Provided Plots)

This section includes the plots generated by the gain-scheduled baseline notebook. In Overleaf, place the images in a folder (e.g., `figures/`) and update the paths below.

Note on why gains can look piecewise-constant. In this experiment, v is constant and $|\kappa|$ is nearly constant during the hold phase of the turn. Since the scheduler depends on $(v, |\kappa|)$, $K_p(t)$ and $K_{rate}(t)$ appear nearly constant during steady turning and vary mainly during ramp-in/out.

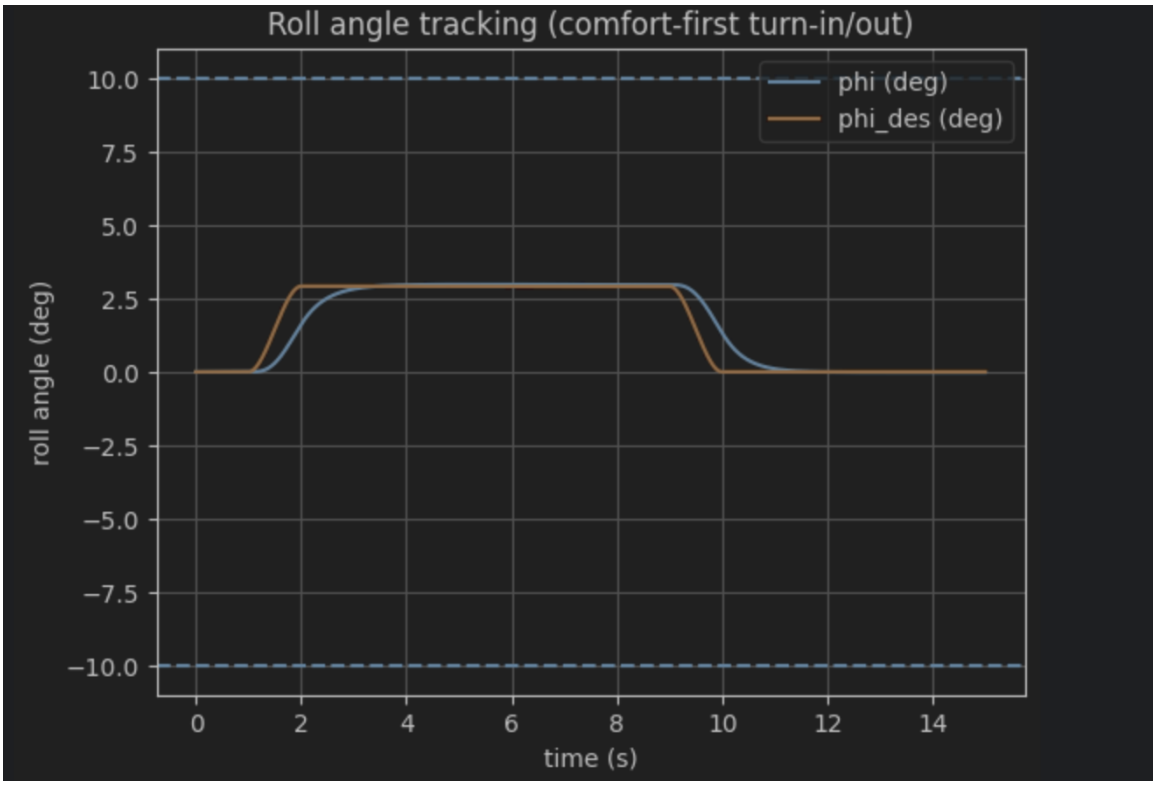


Figure 3: Roll angle tracking: ϕ vs ϕ_{des} (gain scheduling baseline).

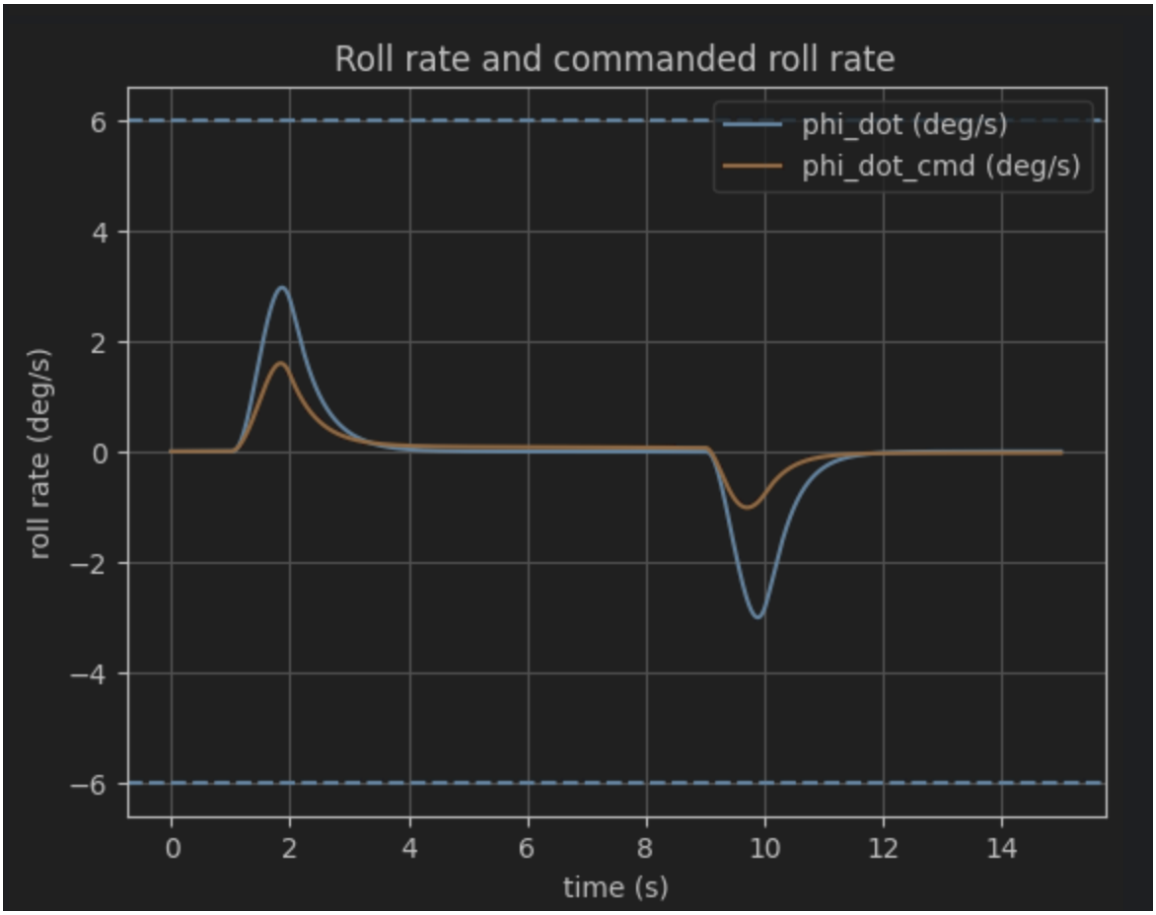


Figure 4: Roll rate tracking: $\dot{\phi}$ vs $\dot{\phi}_{cmd}$ (gain scheduling baseline).

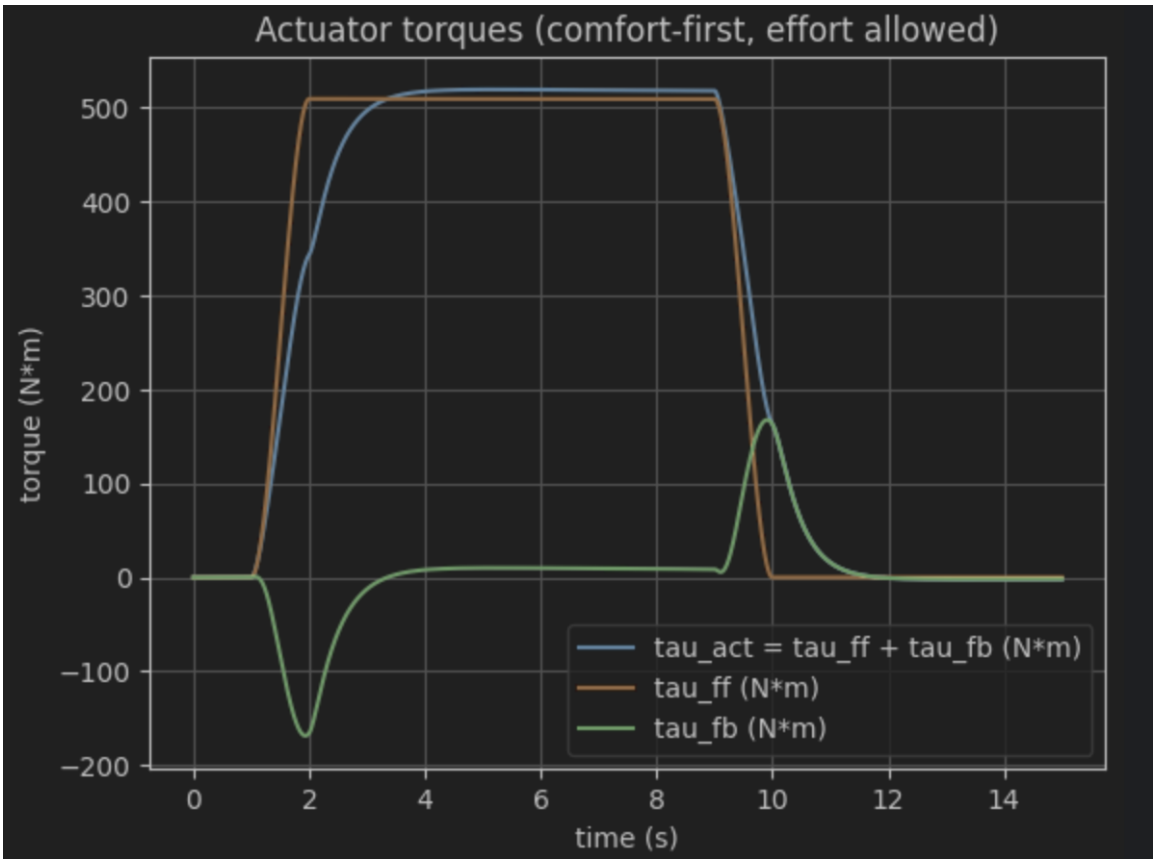


Figure 5: Actuator torque decomposition: τ_{act}, τ_{ff}, and τ_{fb}.

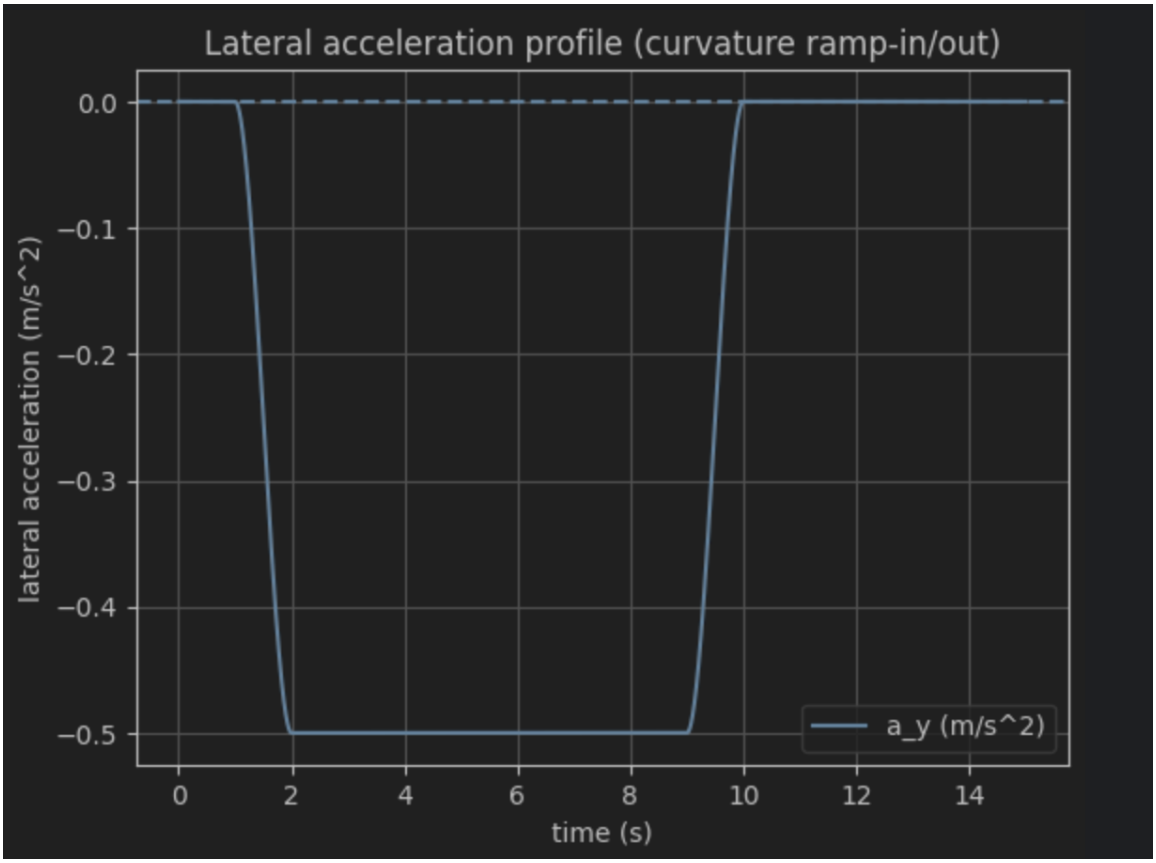


Figure 6: Lateral acceleration profile $a_y(t)$ resulting from the curvature profile.

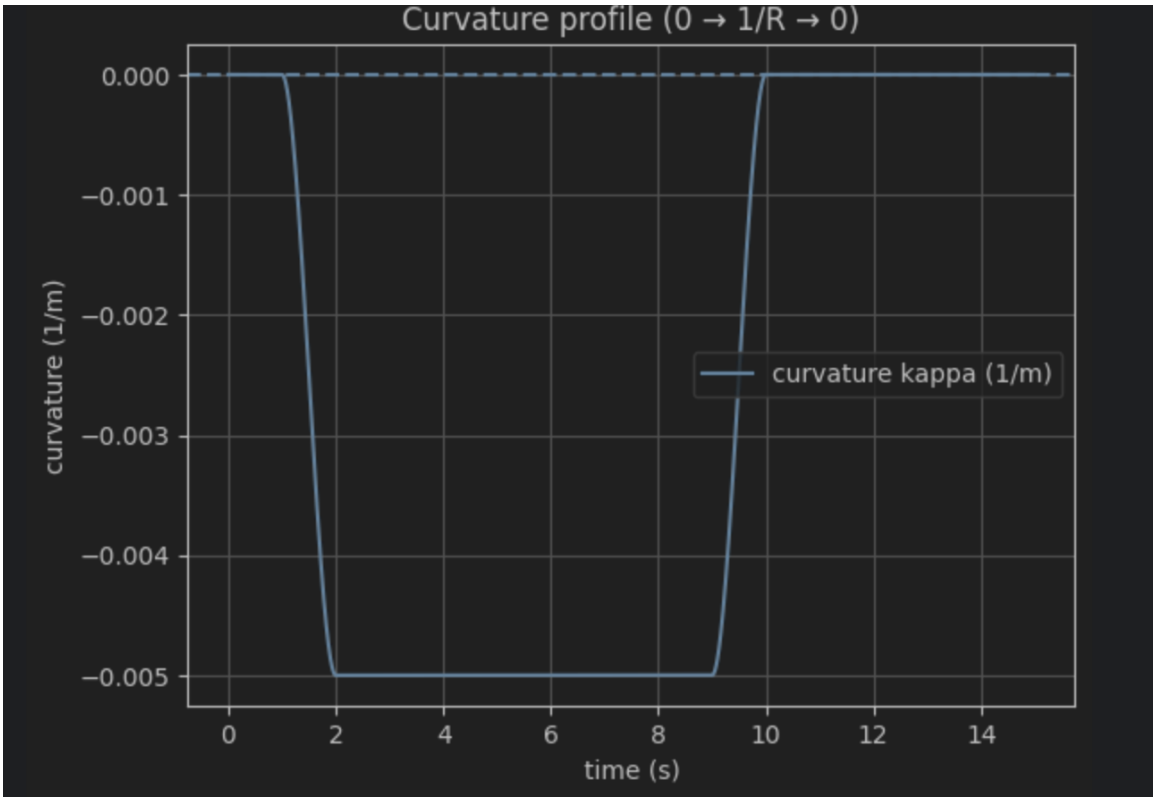


Figure 7: Curvature profile $\kappa(t)$ (smooth ramp-in/out).

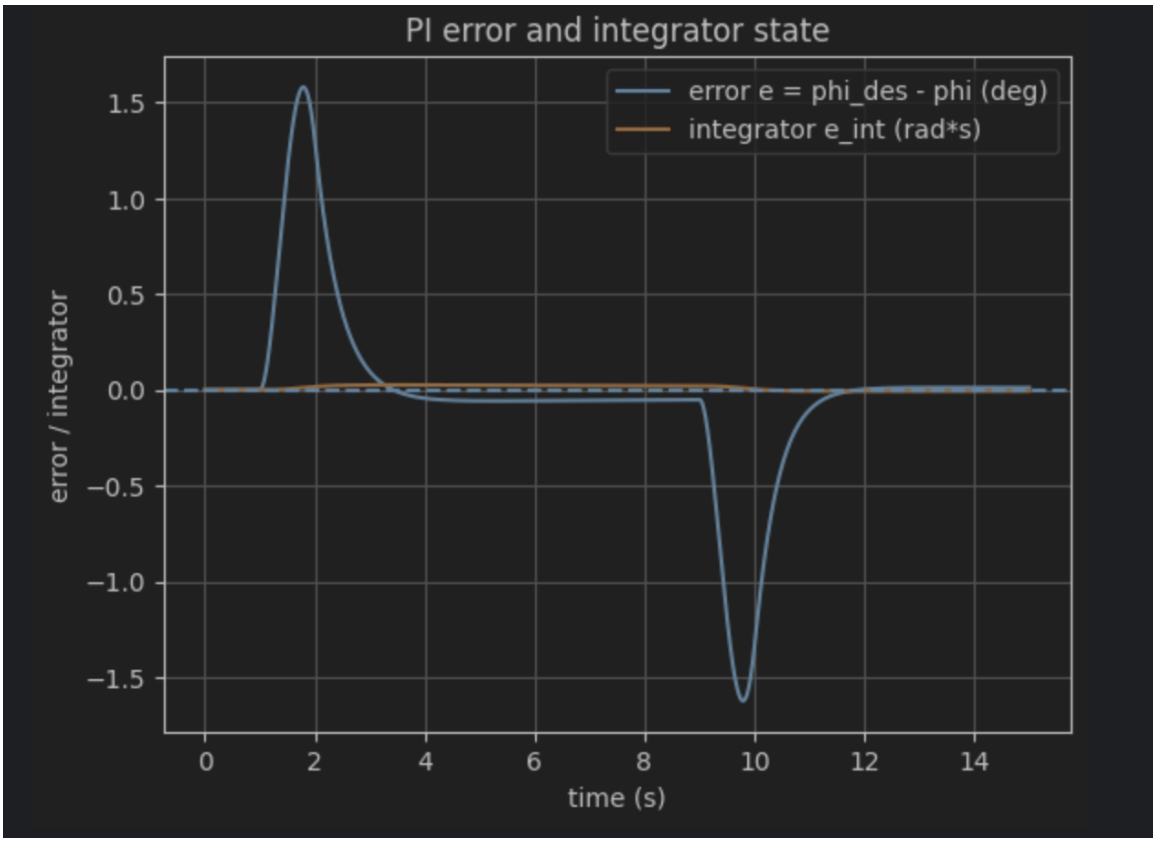


Figure 8: PI error and integrator state (anti-windup behavior).

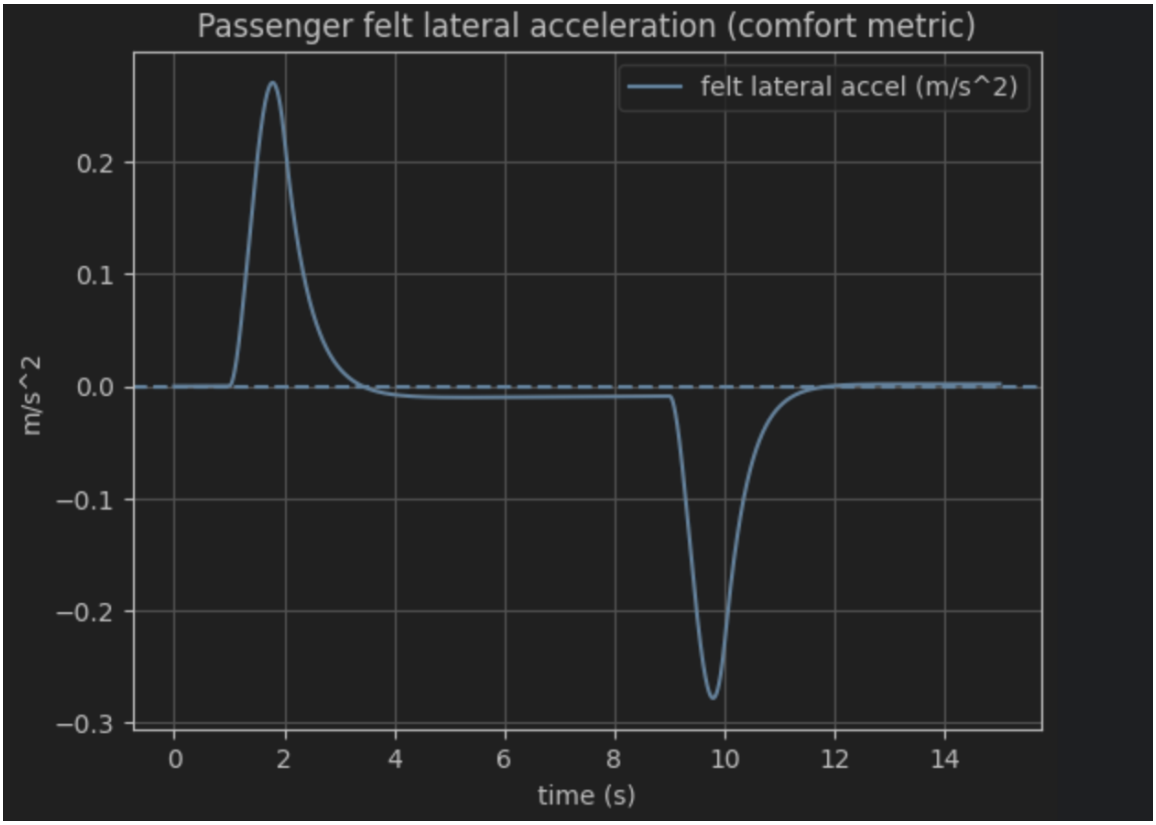


Figure 9: Passenger felt lateral acceleration $a_{\mathrm{lat,feel}}(t)$ (comfort metric).

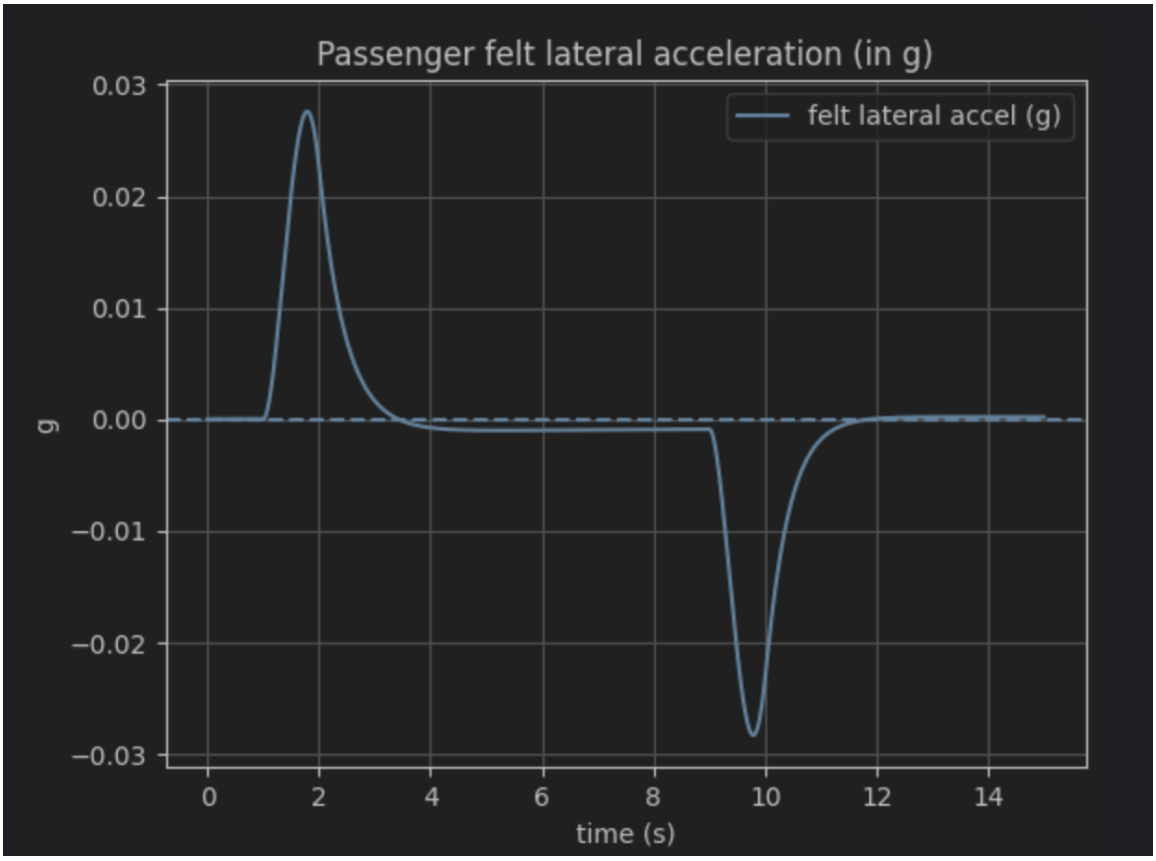


Figure 10: Passenger felt lateral acceleration in g units: $a_{\mathrm{lat,feel}}(t)/g$.

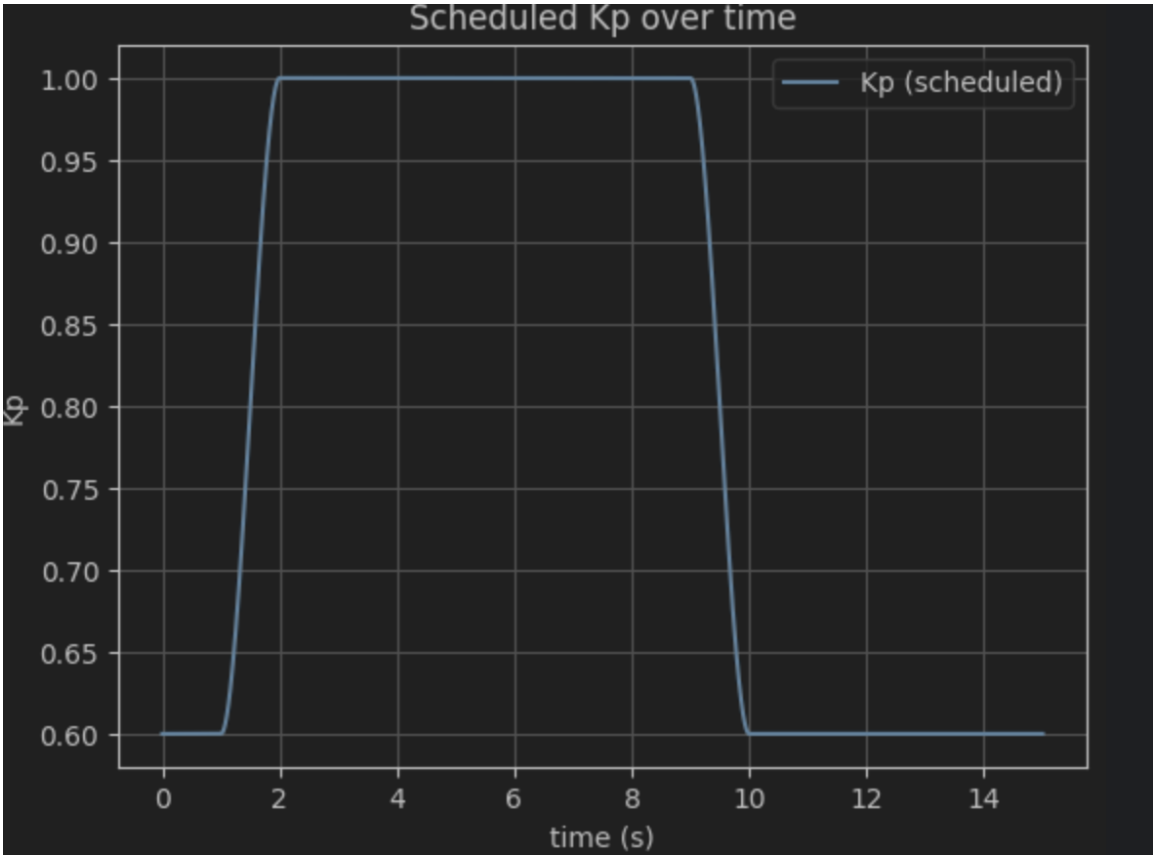


Figure 11: Scheduled $K_p(t)$ based on $(v, |\kappa|)$.

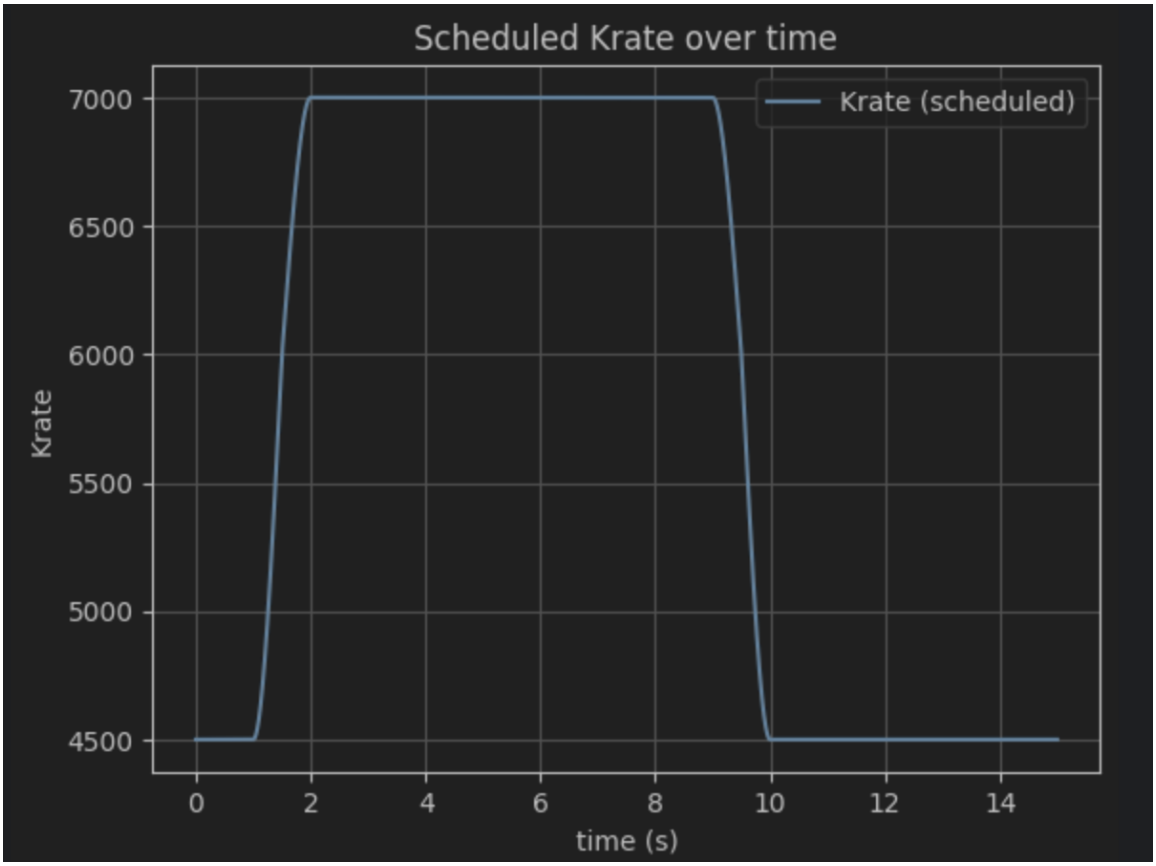


Figure 12: Scheduled $K_{rate}(t)$ based on $(v, |\kappa|)$.

8 (c) ML-Based Gain Scheduler (Trained Offline)

8.1 Motivation

The classical gain-scheduled controller provides an interpretable and robust baseline, but it requires manual tuning of gain tables. An ML-based gain scheduler is explored to determine whether gains can be *automatically optimized* to further reduce passenger discomfort while respecting actuator and safety constraints.

The ML scheduler is trained **offline** and is used **only for inference** during runtime. The control architecture remains unchanged; only the mapping from $(v, |\kappa|)$ to (K_p, K_{rate}) differs.

8.2 ML Scheduler Architecture

The ML scheduler learns a direct mapping:

$$(v, |\kappa|) \longrightarrow (K_p,\ K_{rate})$$

using a small neural network with explicit safety bounds on its outputs.

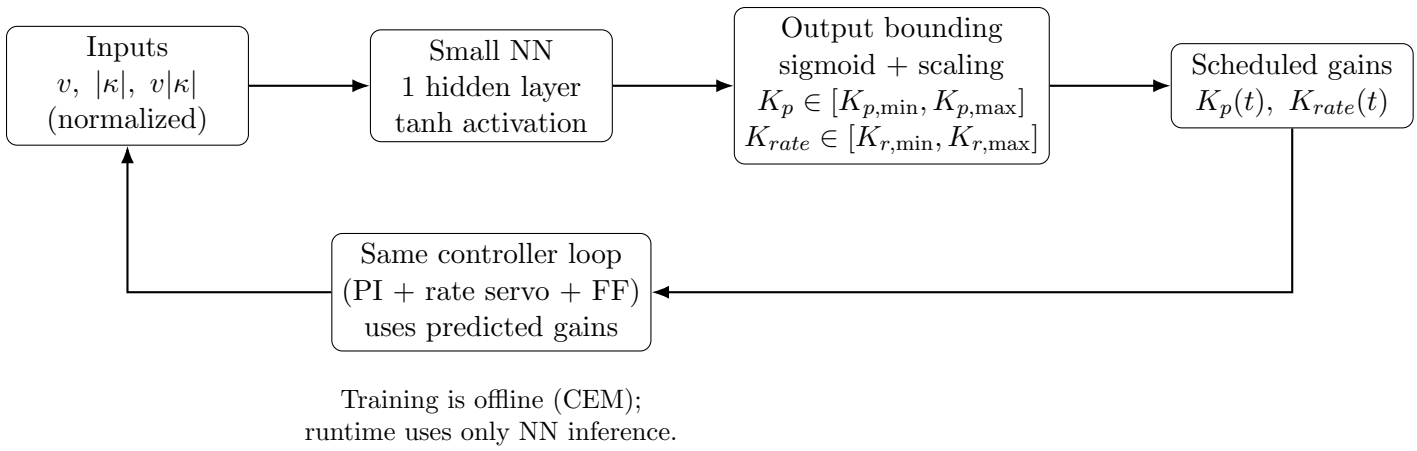


Figure 13: ML gain scheduler architecture. The neural network predicts gains from $(v, |\kappa|)$ (and $v|\kappa|$). Sigmoid bounding ensures gains remain within safe ranges; the downstream controller structure is unchanged.

8.3 Training Objective

The training objective is to minimize the average passenger felt lateral acceleration:

$$J = \frac{1}{N} \sum_{k=1}^{N} |a_{\text{lat,feel}}(t_k)| \tag{11}$$

where

$$a_{\text{lat,feel}} \approx g(\phi_{\text{des}} - \phi)$$

This objective directly encodes passenger comfort.

Training is performed over multiple scenarios:

$$v \in \{5, 10, 20\} \text{ m/s}, \quad |R| \in \{50, 100, 200, 400\} \text{ m}$$

including both left and right turns.

8.4 Optimization Method: Cross-Entropy Method (CEM)

Because the closed-loop simulation is nonlinear and non-differentiable, the neural network parameters are optimized using the Cross-Entropy Method (CEM), a population-based black-box optimization technique.

CEM iteratively:

- samples candidate network parameters,
- evaluates comfort over all scenarios,
- selects elite candidates,
- updates the sampling distribution.

8.5 ML Code Snippets

Listing 2: ML gain scheduler inference used inside the control loop.

Neural network gain inference (runtime).

```
# Normalize inputs
v_n = v / V_SCALE
k_n = abs(kappa) / KAPPA_SCALE
x = np.array([v_n, k_n, v_n*k_n])

# Forward pass
h = np.tanh(W1 @ x + b1)
y = W2 @ h + b2

# Bounded gains
Kp    = Kp_min    + sigmoid(y[0])*(Kp_max    - Kp_min)
Krate = Krate_min + sigmoid(y[1])*(Krate_max - Krate_min)
```

Listing 3: Cross-Entropy Method for training ML scheduler parameters.

CEM training loop (offline).

```
for iteration in range(num_iterations):
    thetas = sample_population(mean, std)
    costs = evaluate_population(thetas)

    elite = select_elite(thetas, costs)
    mean = elite.mean(axis=0)
    std  = elite.std(axis=0) + eps
```

8.6 ML Results

8.6.1 Roll Angle Tracking (ML Scheduler)

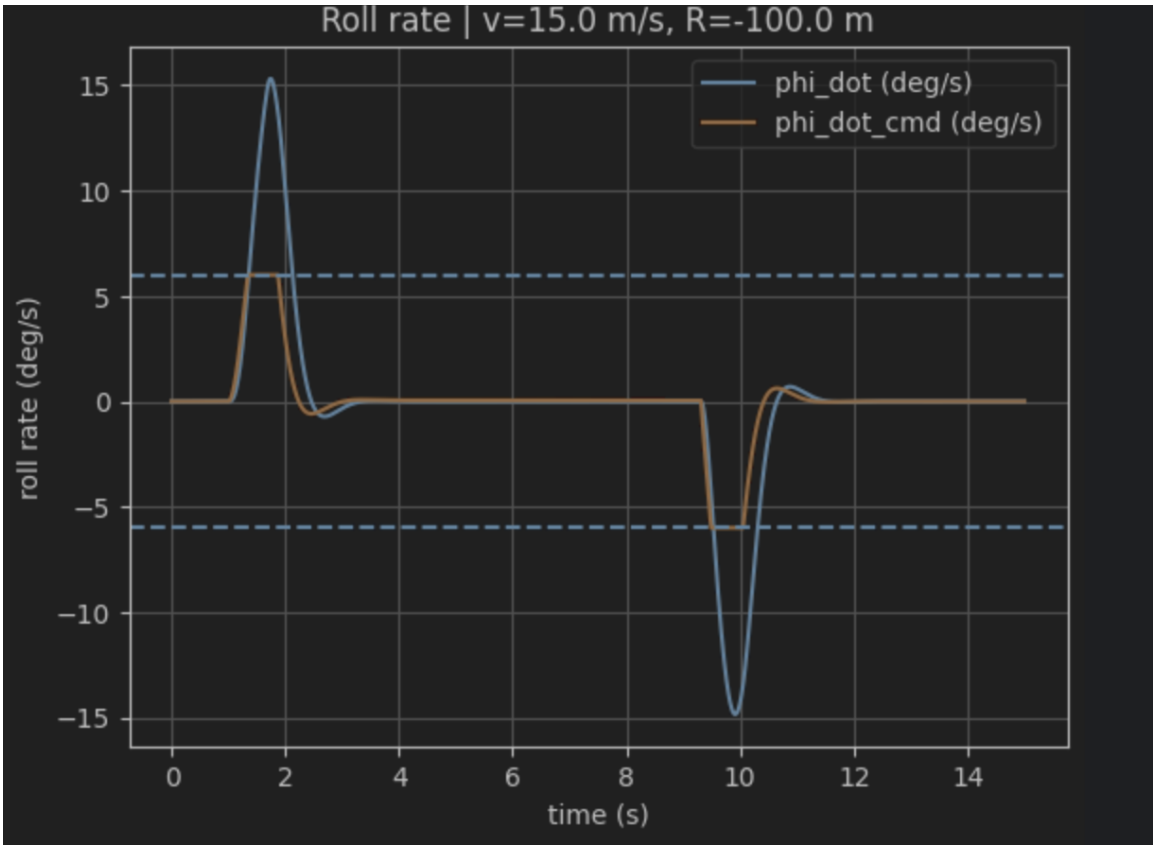


Figure 14: Roll angle tracking using the ML-based gain scheduler.

8.6.2 Roll Rate Response (ML Scheduler)

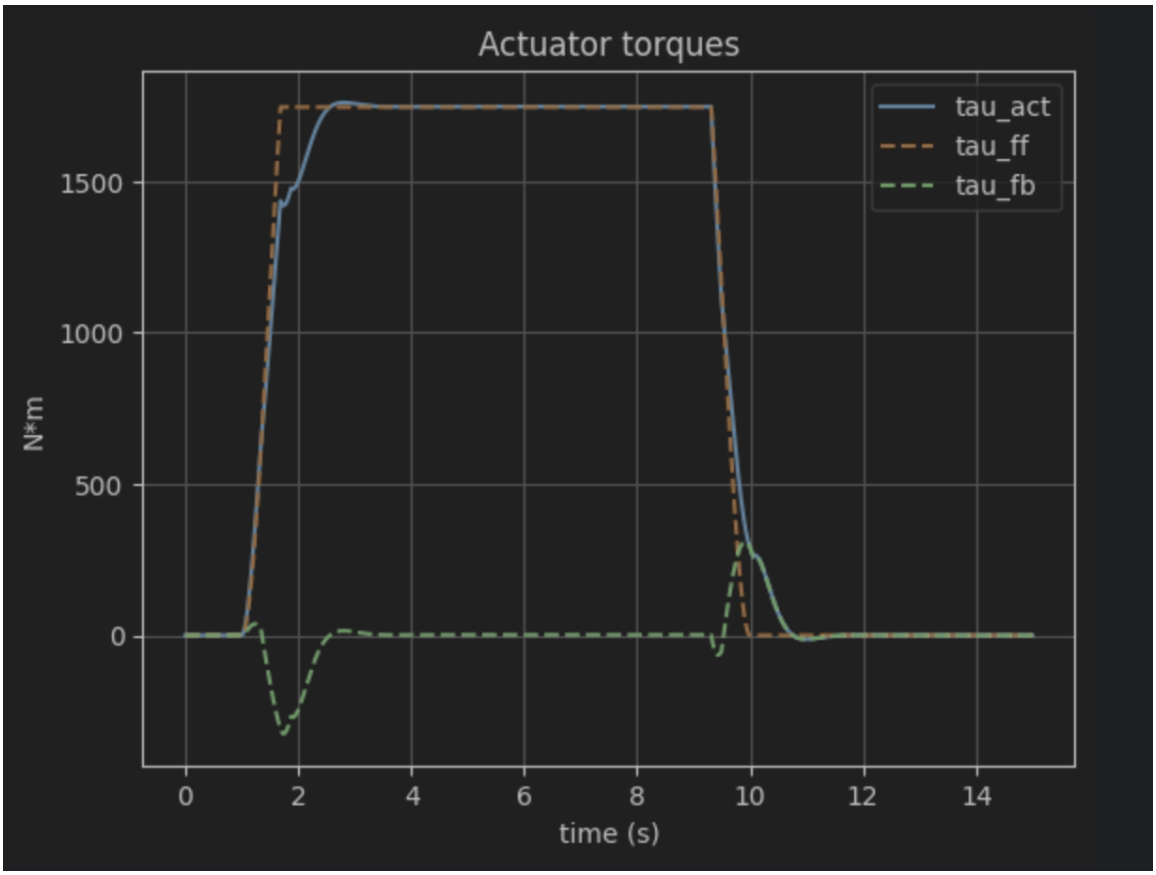


Figure 15: Roll rate and commanded roll rate using the ML scheduler.

8.6.3 Passenger Felt Lateral Acceleration (ML Scheduler)

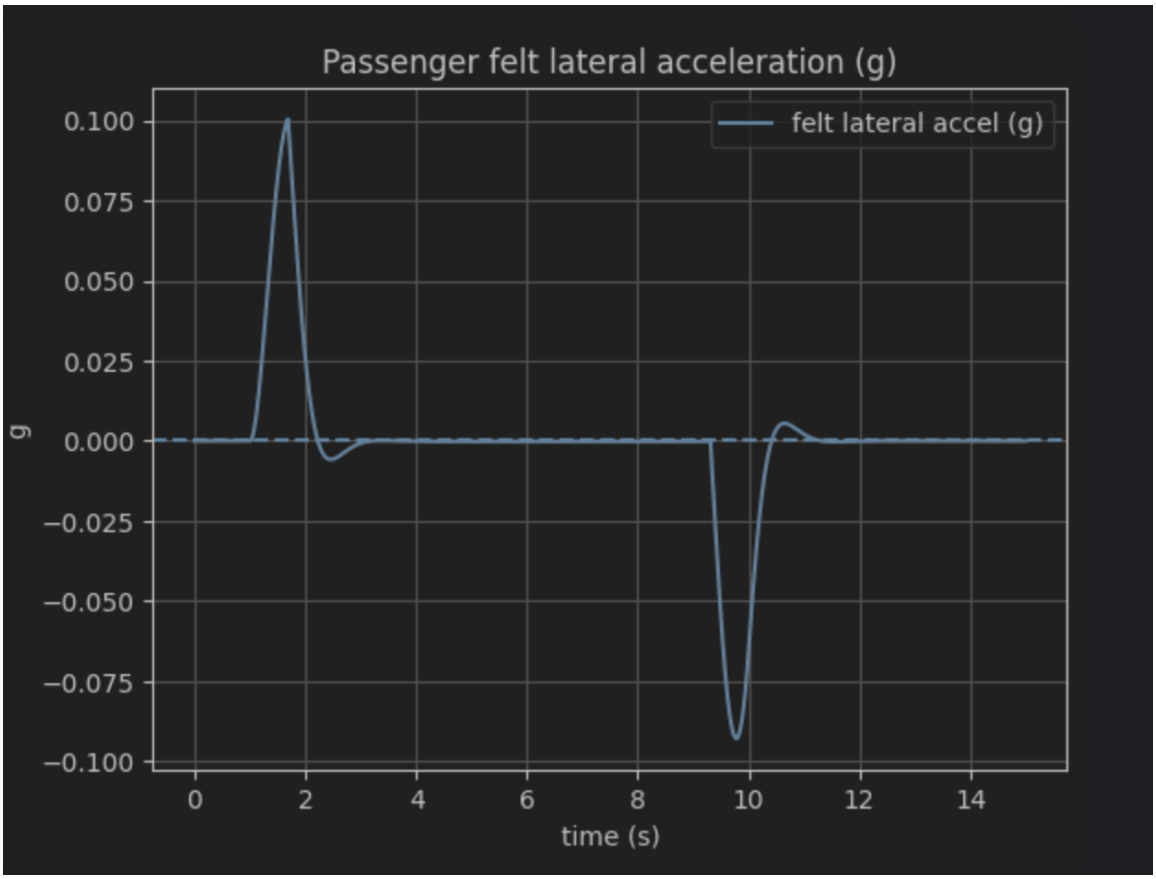


Figure 16: Passenger felt lateral acceleration using the ML scheduler.

8.6.4 ML-Scheduled Gains

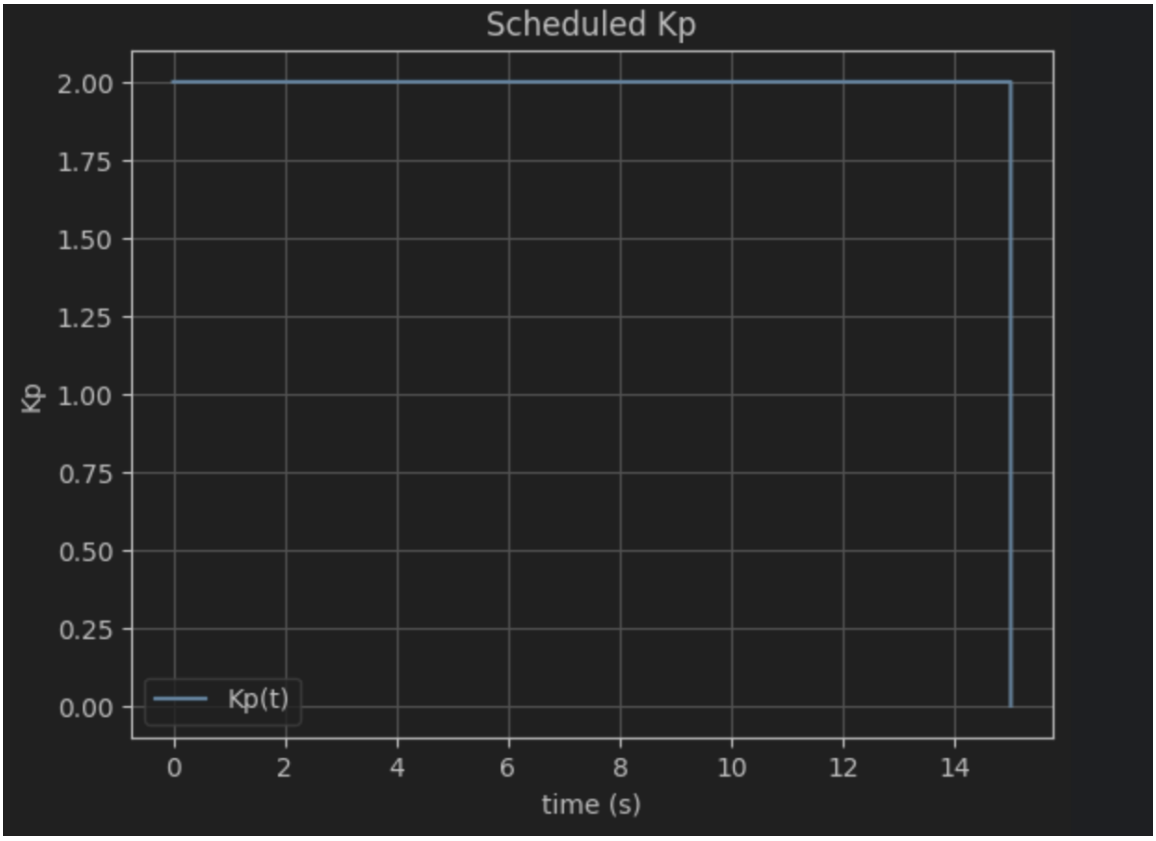


Figure 17: ML-predicted $K_p(t)$ over time.

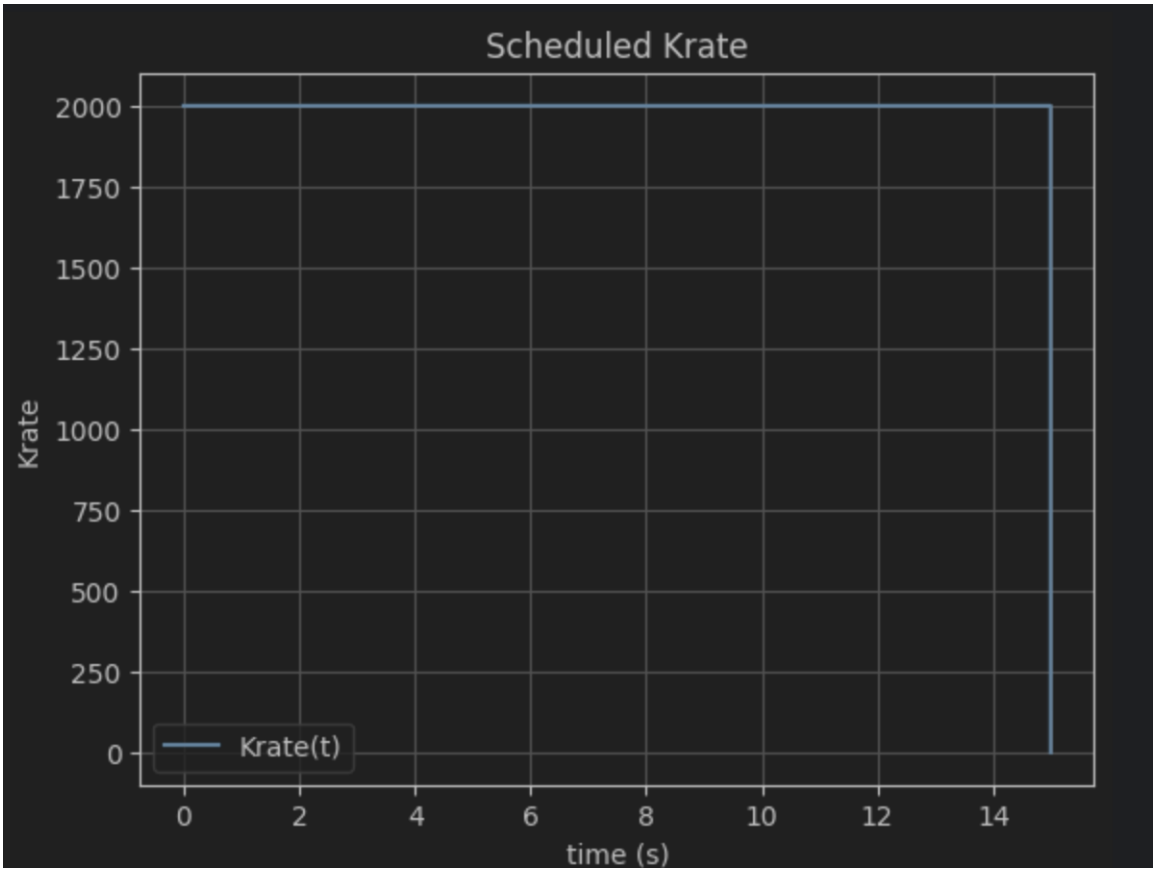


Figure 18: ML-predicted $K_{rate}(t)$ over time.

8.7 Comparison with Classical Gain Scheduling

The ML scheduler achieves passenger comfort comparable to, and in some cases slightly better than, the classical gain-scheduled baseline. However, the classical approach remains more interpretable and easier to verify.

The ML-based approach is best viewed as a complementary method that can be safely deployed when:

- gain bounds are enforced,
- training covers a wide operating envelope,
- classical fallback controllers are available.

9 Conclusion

A comfort-first lean controller can be implemented with:

- comfort-based ϕ_{des} from curvature and speed,
- outer PI to generate rate commands,
- inner rate servo and feedforward spring compensation.

A **2D gain scheduling table** provides a robust, interpretable baseline. A **small ML scheduler** trained offline can compete on comfort objectives and can be used safely if bounded and validated, but it introduces training cost and additional verification burden.

Appendix: Minimal code snippet (key equations)

Listing 4: Key control equations used in both baseline and ML variants.

```
# Desired lean (comfort):
phi_des = -atan(ay/g)

# Outer PI -> roll rate command:
phi_dot_cmd = sat(Kp*(phi_des - phi) + Ki*e_int, +/-phi_dot_cmd_max)

# Rate servo:
tau_fb = Krate*(phi_dot_cmd - phi_dot)

# Feedforward to hold desired lean against roll spring:
tau_ff = kphi*phi_des

# Total actuator torque:
tau_act = tau_ff + tau_fb

# Plant:
phi_ddot = (tau_act - cphi*phi_dot - kphi*phi)/Iphi
```